ZeroFault API — how it works

Since AI 2026 · ZeroFault · server/ · 07 November 2026

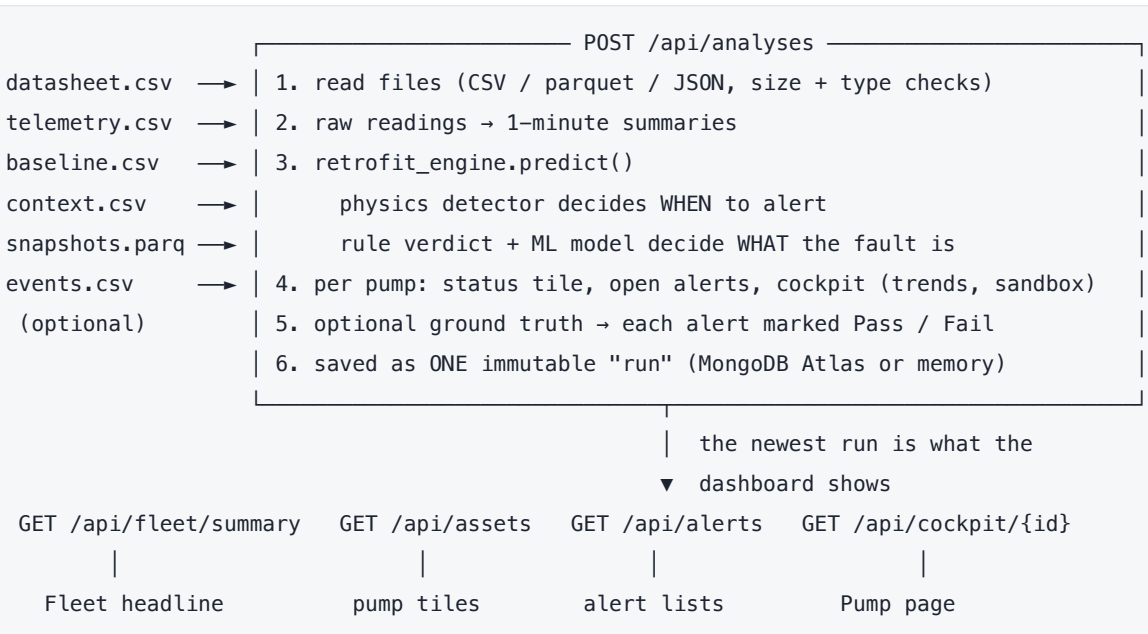
The REST API behind the operator dashboard. It runs our trained hybrid model (Joice's physics rules + the ML diagnostician in `retrofit_engine/`) on uploaded pump data, stores each result, and serves it to the Fleet and Pump screens.

- Base URL (local): `http://localhost:8000`
- Interactive docs: `http://localhost:8000/docs` (Swagger), `http://localhost:8000/redoc`
- All responses are JSON unless noted. Times are ISO-8601 UTC (`2025-08-12T19:27:00Z`).
- The API is **read-only towards the pumps**: it never writes to a PLC or changes a setting.

Contents

1. The big picture
2. Code map
3. Endpoints at a glance
4. Endpoint details
5. Examples
6. Severity rules
7. Mitigation sandbox (cockpit "What happens next?")
8. Storage
9. Settings
10. Troubleshooting

1. The big picture



Key ideas:

Idea	What it means
Run	One model execution over one set of uploaded files. Has a `run_id`. Never changed after it is saved.
Latest run	All dashboard endpoints read the most recent run. A new upload instantly becomes what everyone sees.
Seed run	On startup, if the store is empty, `server/seed/demo_run.json` is loaded so the dashboard is never blank.
Open alert	An alert raised within `ACTIVE_ALERT_HOURS` (default 48 h) before the pump's last reading. Drives the pump's status.

Idea	What it means
Severity	`act_now` · `attention` · `sensor` · `ok` — from the urgency of the pump's most severe open alert.

2. Code map

```
server/src/
├── main.py           app, CORS, lifespan (open store, load seed), /health, mounts routers
├── settings.py       environment variables → Settings
├── engine.py         imports retrofit_engine (repo root); API still starts if it is missing
├── analysis.py       run_analysis(): calls the model and builds tiles / cockpits / CSV
├── sandbox.py        what-if speed derate (pump affinity laws)
├── fault_guide.py    plain names, urgency, steps per fault (data/fault_actions.json)
├── db.py             RunStore: MongoRunStore (Atlas) or MemoryRunStore
├── deps.py           FastAPI dependencies (store, settings) — overridden in tests
├── models.py         Pydantic request/response models
├── sample_data.py    static retrofit scorecard
└── routers/
    ├── dashboard.py  /api/alerts, /api/assets, /api/fleet/summary, /api/cockpit, /api/scorecard,
/api/model
    └── analyses.py   /api/analyses (upload + run list + CSV export)
```

3. Endpoints at a glance

Method	Path	Used by	Purpose
GET	`/health`	App start-up	Is the API up, is the DB connected, which model is loaded
GET	`/api/fleet/summary`	Fleet screen	Headline counts
GET	`/api/assets`	Fleet screen	One status tile per pump
GET	`/api/alerts`	Fleet + Pump screens	Alerts of the latest run (filterable)
GET	`/api/alerts/{asset_id}`	—	Alerts of one pump (404 if none)
GET	`/api/cockpit/{asset_id}`	Pump screen	Diagnosis, evidence, steps, trends, sandbox
GET	`/api/scorecard`	Scorecard screen	Retrofit business case (static)
GET	`/api/model`	Upload dialog	Model version, required columns, upload limit
POST	`/api/analyses`	Upload dialog	Upload files → run model → new latest run
GET	`/api/analyses`	—	Recent runs, newest first
GET	`/api/analyses/latest`	App (run strip)	The run the dashboard is showing
GET	`/api/analyses/{run_id}`	—	One run's summary
GET	`/api/analyses/{run_id}/alerts.csv`	—	Alerts in the organisers' template CSV
GET	`/retrofit/info`	—	Kaleb's stateless engine info
POST	`/retrofit/analyze`, `/retrofit/analyze/csv`	—	Kaleb's stateless engine (result not stored)

4. Endpoint details

`GET /health`

```
{ "status": "ok", "service": "since-ai-server", "database": "connected", "model": "hybrid-v3-001" }
```

`database` is `connected` (Atlas) or `disabled` (memory). `model` is `unavailable` when `retrofit_engine` could not load — dashboard reads still work, uploads return 503.

`GET /api/fleet/summary`

```
{
  "vessel": "4 sites · Fleet overview",
  "watch": "Model hybrid-v3-001 · read-only",
  "critical": 2,
  "actionable": 6,
  "sensor_faults": 1,
  "availability_pct": 22.2,
  "healthy": 2,
  "total": 9,
  "nuisance_suppressed_pct": null,
  "run_id": "e001660ed19b433db16a30d8e1e01ec4",
  "model_version": "hybrid-v3-001",
  "data_end_utc": "2025-08-18T00:00:00Z"
}
```

Field	Rule
`critical`	pumps with severity `act_now`
`actionable`	`act_now` + `attention`
`sensor_faults`	pumps whose worst open alert is a sensor problem
`availability_pct`	`healthy / total × 100`
`nuisance_suppressed_pct`	not computed by the model yet → `null`

`GET /api/assets`

One tile per pump in the latest run.

```
[
  {
    "asset_id": "PUMP-01",
    "site": "Vessel A (RoPax, fictional)",
    "system": "Seawater cooling, pump No. 1",
    "tier": "T1",
    "status": "alert",
    "mode": "RUNNING",
    "load_pct": 19.9,
    "open_alerts": 2,
    "fault_summary": "Inlet filter is blocked",
    "rul_hours": null,
    "severity": "attention",
    "last_data_utc": "2025-08-18T00:00:00Z"
  }
]
```


How each field is filled:

Field	Source
`site`, `system`	datasheet columns `site`, `system` (fallback `profile`, then `<control> pump`)
`tier`	`T0` / `T1` / `T2` from the datasheet or the sensors present in the telemetry
`mode`	most common `operating_mode` in the pump's last 15 minutes
`load_pct`	power / rated kW → else current / rated A → else speed / rated rpm (0 when not running)
`open_alerts`	alerts within `ACTIVE_ALERT_HOURS` of `last_data_utc`
`severity`	urgency of the most severe open alert (see §6); `ok` when none
`status`	`alert` (act_now / attention) · `sensor_issue` (sensor) · `healthy` (ok) — kept for older UI code
`fault_summary`	plain name of the most severe open alert
`rul_hours`	`null` — the model does not estimate remaining life

`GET /api/alerts`

Query parameters (both optional):

Name	Values
`asset_id`	e.g. `PUMP-07`
`event_class`	`MACHINE` or `SENSOR` (anything else → 422)
`severity`	repeatable: `SEVERE`, `MODERATE`, `MINIMAL`, `LOW_CONFIDENCE`, e.g. `?severity=SEVERE&severity=MODERATE`

```
[
  {
    "asset_id": "PUMP-01",
    "alert_time_utc": "2025-08-12T19:27:00Z",
    "event_class": "MACHINE",
    "fault_type": "SUCTION_STRAINER_CLOGGING",
    "confidence": 0.827,
    "evidence": "suction_pressure_kpa z=-6.4; head +0% power -4% vs reference | ML agrees (0.83)",
    "recommended_action": "Inspect / clean suction strainer.",
    "true_fault_type": "SUCTION_STRAINER_CLOGGING",
    "correct": true,
    "severity": "SEVERE"
  }
]
```


`severity` is a per-alert triage band derived by the server (`src/severity.py`, in the shared contract): `SEVERE` = MACHINE alert with confidence ≥ 0.75 · `MODERATE` = other alerts ≥ 0.50 · `MINIMAL` = below 0.50 · `LOW_CONFIDENCE` = fault type `UNKNOWN`. It never changes the detector's fields or the benchmark CSV. Not to be confused with a pump's `severity` in `/api/assets` (act_now / attention / sensor / ok).

- Shape = `AlertPrediction` from `shared/contracts.schema.json` (+ the existing optional `true_fault_type`, `correct`, filled only when ground truth was uploaded).
- Sorted by pump, then time (oldest first).
- `fault_type` is one of the 13 benchmark faults or `UNKNOWN`.
- `confidence` is the calibrated model confidence (0–1). When the ML model agrees with the physics rule it is the higher of the two; when the ML model overrides the rule it is the ML confidence.

`GET /api/alerts/{asset_id}` returns the same list for one pump, or **404** if it has no alerts.

`GET /api/cockpit/{asset_id}`

Everything the Pump page needs. **404** for an unknown pump.

```
{
  "asset_id": "PUMP-01",
  "title": "Seawater cooling, pump No. 1",
  "diagnosis": "Inlet filter is blocked",
  "fault_type": "SUCTION_STRAINER_CLOGGING",
  "event_class": "MACHINE",
  "confidence": 0.75,
  "severity": "SEVERE",
  "safe_window": "Hours to days. Act now if cavitation alarms appear on the same pump.",
  "evidence": [
    { "channel": "Suction pressure", "sigma": -9.4, "note": "Below this pump's own normal" }
  ],
  "sop": [
    "Switch to the standby pump if one is available.",
    "Inspect and clean the suction strainer.",
    "After cleaning, check that suction pressure is back to normal."
  ],
  "trends": [
    { "name": "Suction pressure", "unit": "kPa", "threshold": null,
      "points": [ { "t": "2025-08-12T06:00:00Z", "value": 48.21 }, "… up to 168 hourly points" ] }
  ],
  "sandbox_default_speed_pct": 100,
  "npsh_at_100": 11.95,
  "npsh_at_88": 12.27,
  "risk_at_100": 0.75,
  "risk_at_88": 0.582,
  "energy_delta_eur_day": -12.88,
  "cooling_margin_pct": 110,
  "cooling_safe_flow_m3h": 252.4,
  "rated_rpm": 1780,
  "sandbox_note": "Affinity-law estimate from the last running hour. … Assumed: no flow meter: flow
estimated from speed."
}
```


Part	How it is built
`diagnosis`, `fault_type`	the most severe open alert (most recent when tied); `"No active fault"` when none
`safe_window`, `sop`	urgency text and steps from `fault_guide` for that fault
`evidence`	channel z-scores parsed from the alert's evidence text (`sigma` = deviations from the pump's own normal)
`trends`	up to 3 series: the channels named in the evidence first, then defaults (vibration/bearing for mechanical faults, pressures/flow/power for hydraulic ones). Hourly means, ≤ 168 points. `threshold` = datasheet `vibration_alarm_mm_s` for vibration RMS.
sandbox fields	see §7

`GET /api/scorecard`

Static retrofit business case (BOM, economics, T0 vs T1 capabilities). Not computed from runs.

`GET /api/model`

```
{
  "available": true,
  "model_version": "hybrid-v3-001",
  "physics_version": "joice-v3",
  "classes": ["AIR_ENTRAINMENT", "BEARING_WEAR", "…", "NO_FAULT", "…"],
  "calibrated": true,
  "reason": null,
  "required_datasheet_columns": ["asset_id", "control", "motor_rated_rpm", "pump_rated_flow_m3_h",
                                 "pump_rated_head_m", "pump_shutoff_head_m", "npsh3_at_bep_m",
"fluid"],
  "min_history_hours": 24,
  "max_upload_mb": 200
}
```

When the engine cannot load: `{"available": false, "reason": "<why>", "max_upload_mb": 200, …}`.

`POST /api/analyses` — upload and run the model

`multipart/form-data`:

Field	Required	Format	Content
`datasheet`	**yes**	csv / parquet / json	one row per pump; required columns from `/api/model`. Optional: `site, system, profile, motor_rated_kw, motor_rated_current_a, bearing_bpfo_order, impeller_vanes, vibration_alarm_mm_s, min_speed_ratio, instrumentation_tier, channels_available`
`telemetry`	**yes**	csv / parquet / json	the period to check. Either raw readings `asset_id, timestamp_utc, <sensor columns>` (any rate) or 1-minute summaries `asset_id, window_start_utc, <sensor>__mean/std/min/max/count, operating_mode, running_fraction, rows`
`baseline`	no	same as telemetry	≥ 24 h of healthy running just before the telemetry. Without it the first ~24 h of telemetry are "normal" and get no alerts.
`context`	no	csv	`asset_id, timestamp_utc, sea_state_hs_m, suction_static_head_m`
`snapshots`	no	parquet	vibration waveforms `asset_id, timestamp_utc, sample_rate_hz, rpm_at_capture, samples` (improves bearing / misalignment / imbalance)
`events`	no	csv / json	ground truth `asset_id, fault_type, onset_utc[, end_utc]` → alerts get `true_fault_type` + `correct`
`use_ml`	no	form field	`true` (default) = rules + ML; `false` = physics rules only

Sensor columns: `motor_speed_rpm, motor_current_a, motor_power_kw, suction_pressure_kpa, discharge_pressure_kpa, flow_m3_h, fluid_temp_c, bearing_temp_c, motor_winding_temp_c, vibration_rms_mm_s, vibration_accel_rms_g, vibration_crest_factor, ambient_temp_c, valve_position_pct` — send the ones the pump has.

Response `201 Created` — the run summary (it is now the latest run):

```
{
  "run_id": "e6a188e005694c14bdc1a7540e3d6bb4",
  "created_utc": "2026-11-07T19:30:43Z",
  "source": "upload",
  "label": "datasheet.csv + telemetry.csv",
  "model_version": "hybrid-v3-001",
  "physics_version": "joice-v3",
  "use_ml": true,
  "pumps": 2,
  "alerts": 13,
  "ml_overrides": 1,
  "by_fault_type": { "MISALIGNMENT": 3, "SUCTION_STRAINER_CLOGGING": 2, "BEARING_WEAR": 2, "…": 1 },
  "data_start_utc": "2025-09-09T06:00:00Z",
  "data_end_utc": "2026-02-16T00:00:00Z",
  "processing_seconds": 0.75,
  "scored": false,
  "notes": ["datasheet: instrumentation_tier derived from the sensors present in the telemetry"]
}
```


`notes` are warnings worth showing (short baseline, missing snapshots, derived columns, pumps skipped…).

Errors — body is always `{"detail": "<message for the user>"}`:

Status	When	Example `detail`
415	file is not csv / parquet / json	`telemetry file 'data.xlsx': use CSV, parquet or JSON`
413	a file is over `MAX_UPLOAD_MB`	`telemetry file 'big.csv' is larger than 200 MB`
422	empty / unreadable file, missing or bad columns, unknown control type, no pump in both files	`datasheet is missing columns ['control', …]; required: […]`
422	a required form field is missing	FastAPI validation list
503	the model engine is not installed / failed to load	`model engine unavailable: …`

Typical time: under 1 s for 2 pumps × 5 days of 1-minute data, ~3 s for 9 pumps × 7 days. Analyses run one at a time (a lock) so parallel uploads cannot starve the API; raw 1 Hz files take longer because they are aggregated first.

Run history

- `GET /api/analyses?limit=20` → list of run summaries, newest first (`limit` 1–100).
- `GET /api/analyses/latest` → the run the dashboard shows (404 before any run exists).
- `GET /api/analyses/{run_id}` → one summary (404 if unknown).
- `GET /api/analyses/{run_id}/alerts.csv` → `text/csv` download with the organisers' template columns `asset_id,alert_time_utc,event_class,fault_type,confidence,evidence,recommended_action`.

5. Examples

curl — upload and read back

```
curl -X POST http://localhost:8000/api/analyses \
  -F datasheet=@datasheet.csv -F telemetry=@telemetry.csv -F baseline=@baseline.csv \
  -F context=@context.csv -F snapshots=@snapshots.parquet -F use_ml=true

curl http://localhost:8000/api/fleet/summary
curl "http://localhost:8000/api/alerts?asset_id=PUMP-09&event_class=MACHINE"
curl http://localhost:8000/api/cockpit/PUMP-09
curl -OJ http://localhost:8000/api/analyses/<run_id>/alerts.csv
```

Demo upload files: `python -m retrofit_engine.tools.make_demo_files --data <dataset>/public --pumps PUMP-08 PUMP-09 --out demo_files`.

TypeScript (client/src/api/client.ts)

```
const form = new FormData();
form.append("datasheet", datasheetFile);
form.append("telemetry", telemetryFile);
form.append("use_ml", "true");
const run = await createAnalysis(form);         // POST /api/analyses → AnalysisRun
const cockpit = await fetchCockpit("PUMP-09");  // reads the new latest run
// errors throw ApiError { status, message = server's "detail" }
```

6. Severity rules

Each fault type has an urgency level in `src/data/fault_actions.json` (Joice's guide). The pump takes the level of its most severe open alert:

Urgency level	Faults	Dashboard severity
`act_now`	CAVITATION, AIR_ENTRAINMENT	**act_now** (red)
`within_a_day`	SUCTION_STRAINER_CLOGGING, DISCHARGE_RESTRICTION, UNKNOWN	**attention** (orange)
`next_planned_stop`	BEARING_WEAR, MISALIGNMENT, IMBALANCE, IMPELLER_WEAR	**attention**
`check_sensor`	SENSOR_DRIFT, SENSOR_STUCK, SENSOR_CALIBRATION_SHIFT, SENSOR_NOISE_SPIKES, COMMUNICATION_LOSS	**sensor** (blue)
no open alert	—	**ok** (green)

7. Mitigation sandbox (cockpit "What happens next?")

Computed in `src/sandbox.py` from the median of the pump's last running hour and its datasheet, for today's speed (100 %) and 88 % of it. The UI slider interpolates between the two.

Field	Physics
`npsh_at_100`, `npsh_at_88`	NPSH available − NPSH required, in metres. NPSHr ∝ speed²; NPSHa from suction pressure, water vapour pressure (Antoine) and velocity head. Higher = further from cavitation.
`energy_delta_eur_day`	power ∝ speed³ → `(P×0.88³ − P) × 24 h × ENERGY_PRICE_EUR_KWH` (negative = saving)

Field	Physics
`cooling_margin_pct`	flow at 88 % speed vs the share of today's flow the cooled system needs (`COOLING_MIN_FLOW_FRACTION`)
`cooling_safe_flow_m3h`	that minimum flow
`risk_at_100`	model confidence that the machine fault is present (0.05 when healthy / sensor-only)
`risk_at_88`	suction-driven faults scaled by NPSH ratio; bearing / misalignment / imbalance × 0.88²; others × 0.88

Risk is a guide, not a failure probability. `sandbox_note` states this and lists per-pump assumptions (no flow meter, no suction gauge, fixed-speed pump without a drive, 88 % below the drive minimum…).

8. Storage

Mode	When	Behaviour
MongoDB Atlas	`MONGODB_URI` set and reachable	collection `analysis_runs`, one document per run (`_id` = `run_id`), indexed by `summary.created_utc`; recent runs cached in memory
Memory	`MONGODB_URI` empty or Atlas unreachable	runs live until the process restarts

On startup, if there is no run yet, the seed run (`SEED_RUN_PATH`) is saved. Rebuild it after a model update: `python -m scripts.build_seed_run --data <dataset>/public` (see `README.md`).

9. Settings

Variable	Default	Meaning
`MONGODB_URI`, `MONGODB_DB`	empty, `since_ai`	Atlas connection
`CORS_ORIGINS`	localhost + VM IP	comma-separated allowed origins
`MAX_UPLOAD_MB`	200	per uploaded file (the host nginx `client_max_body_size` must allow it too)
`ACTIVE_ALERT_HOURS`	48	window for "open" alerts
`ENERGY_PRICE_EUR_KWH`	0.15	sandbox energy cost
`COOLING_MIN_FLOW_FRACTION`	0.8	sandbox cooling requirement
`SEED_RUN_PATH`	`seed/demo_run.json`	demo run for an empty store

10. Troubleshooting

Symptom	Cause / fix
Dashboard shows "example data" banner	`/health` not reachable — API down or wrong `VITE_API_BASE_URL` / proxy
`/health` says `"model": "unavailable"`	`retrofit_engine/` missing or a dependency (lightgbm needs `libgomp1`) failed; see startup log `Model engine: unavailable — …`

Symptom	Cause / fix
Upload returns 413 from nginx (HTML page, not JSON)	raise `client_max_body_size` on the host nginx
A pump has no alerts at all	less than ~24 h of data and no baseline — the model needs a "normal" period first
Bearing / misalignment rarely detected	upload `snapshots` (vibration waveforms)
`"database": "disabled"` in production	`MONGODB_URI` secret missing or Atlas unreachable — runs are lost on restart